

Infrastructure Inventory Management Tool

Enterprise-grade IT asset inventory platform built with React (Ant Design), Node.js/Express, and PostgreSQL. Designed for direct deployment on Ubuntu server.

Features

- **Dashboard** — total assets, breakdowns by OS / status / location / EOL, missing security tools, recent additions, charts
- **Asset CRUD** — full asset lifecycle with 25+ fields, validation, duplicate detection, encrypted credentials
- **Excel Import / Export** — downloadable template, row-by-row validation, success/failure logs
- **Custom Pages** — admin-defined pages with dynamic fields stored in PostgreSQL JSONB
- **Role-Based Access** — Admin, Asset Manager, Viewer
- **Audit Logs** — every create/update/delete/import/export/login event recorded
- **User & Dropdown Management** — admin tools for users and dropdown master values
- **Swagger API Docs** — </api/docs>

Stack

Layer	Technology
Frontend	React 18 + Vite + Ant Design 5 + React Router
Backend	Node.js 20 + Express 4 + JWT + bcrypt
DB	PostgreSQL 15 (pg driver, JSONB)
Files	exceljs (Excel), pdfkit (PDF)
Docs	swagger-ui-express
Process	systemd
Proxy	nginx

Project Layout

```
Inventory_Tool/  
├── backend/          # Node.js + Express API  
├── frontend/         # React + Ant Design SPA  
├── db/               # PostgreSQL schema + seed data  
├── deploy/           # systemd unit files + nginx config  
├── .env.example  
└── README.md
```

Ubuntu Server Setup (Production)

1. Install prerequisites

```
sudo apt update  
sudo apt install -y curl ca-certificates gnupg nginx postgresql postgresql-contrib  
  
# Node.js 20 (NodeSource)  
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
sudo apt install -y nodejs
```

2. Create PostgreSQL database

```
sudo -u postgres psql <<EOF  
CREATE USER inventory_user WITH PASSWORD 'change-me-strong-pass';  
CREATE DATABASE inventory_db OWNER inventory_user;  
GRANT ALL PRIVILEGES ON DATABASE inventory_db TO inventory_user;  
EOF  
  
# Load schema + seed  
sudo -u postgres psql -d inventory_db -f /opt/inventory/db/schema.sql  
sudo -u postgres psql -d inventory_db -f /opt/inventory/db/seed.sql
```

3. Deploy the code

```
sudo mkdir -p /opt/inventory  
sudo chown -R $USER:$USER /opt/inventory  
# Copy project files (or git clone)  
cp -r ./Inventory_Tool/* /opt/inventory/
```

```
# Backend
cd /opt/inventory/backend
cp .env.example .env
# Edit .env: set DB_PASSWORD, JWT_SECRET, ENCRYPTION_KEY
nano .env
npm ci --omit=dev

# Frontend (build static bundle)
cd /opt/inventory/frontend
cp .env.example .env
# Edit VITE_API_BASE_URL=/api
nano .env
npm ci
npm run build
# Output goes to /opt/inventory/frontend/dist
```

4. systemd service for backend

```
sudo cp /opt/inventory/deploy/inventory-backend.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now inventory-backend
sudo systemctl status inventory-backend
```

5. nginx reverse proxy + static frontend

```
sudo cp /opt/inventory/deploy/nginx-inventory.conf /etc/nginx/sites-available/inventory
sudo ln -s /etc/nginx/sites-available/inventory /etc/nginx/sites-enabled/inventory
sudo nginx -t && sudo systemctl reload nginx
```

Visit `http://<your-server>/`. Login with the seed admin: `admin@example.com` / `Admin@123` (change immediately).

For HTTPS, run `sudo certbot --nginx -d your.domain.com`.

Local Development

```
# Backend
cd backend
cp .env.example .env
npm install
npm run dev                # http://localhost:4000

# Frontend (new terminal)
cd frontend
cp .env.example .env
npm install
npm run dev                # http://localhost:5173
```

Vite proxies `/api/*` to the backend (see `frontend/vite.config.js`).

Roles

Role	Permissions
Admin	Everything — users, dropdowns, custom pages, audit, all CRUD
Asset Manager	Create / edit / delete / import / export assets
Viewer	Read-only access to assets and dashboard

API Surface (highlights)

Method	Path	Purpose
POST	/api/auth/login	Login, returns JWT
GET	/api/dashboard/summary	Dashboard counts + charts
GET	/api/assets	List (search, filter, page)
POST	/api/assets	Create asset
PUT	/api/assets/:id	Update asset
DELETE	/api/assets/:id	Delete asset
POST	/api/assets/import	Import Excel
GET	/api/assets/template	Download Excel template

Method	Path	Purpose
GET	/api/assets/export	Export filtered assets
GET	/api/custom-pages	List custom pages
POST	/api/custom-pages	Create custom page + fields
GET	/api/custom-pages/:id/records	List records
POST	/api/custom-pages/:id/records	Add record (JSONB)
GET	/api/users	(Admin) user management
GET	/api/dropdowns	All dropdown values
GET	/api/audit	(Admin) audit log

Full schema at `/api/docs`.

Security

- Passwords hashed with bcrypt (12 rounds)
- Asset credentials encrypted with AES-256-GCM (`backend/src/utils/crypto.js`)
- JWT signed with `JWT_SECRET` — rotate via env
- Input validation on every route (express-validator)
- helmet + CORS + rate limiting on `/api/auth/login`
- nginx terminates TLS in production (certbot)

Updating the deployed app

```
cd /opt/inventory
git pull                                # or rsync new files
cd backend && npm ci --omit=dev
cd ../frontend && npm ci && npm run build
sudo systemctl restart inventory-backend
sudo systemctl reload nginx
```

License

Internal use.

